

PROCESO SDD CON KIRO

Landing page del evento

Registro paso a paso Proceso SDD con Kiro

Uso de Spec-Driven Development (SDD) en Kiro (IDE de AWS) —
bloque “Practicar” del taller

CONTENIDO DE ESTE DOCUMENTO

- 01 Paso 1 — Spec inicial en el chat de Kiro
- 02 Paso 2 — Secciones, requisitos funcionales y no funcionales
- 03 Paso 3 — Archivos generados por Kiro
- 04 Paso 4 — Visualización de requirements.md
- 05 Paso 5 — Visualización de design.md
- 06 Paso 6 — Generación y ejecución del plan de tareas
- 07 Paso 7 — Visualización de tasks.md y ejecución controlada
- 08 Paso 8 — Cada tarea corre en su propia sesión de chat
- 09 Paso 9 — Todas las tareas ejecutadas: cierre del plan
- 10 Paso 10 — Resultado final: la landing page renderizada



Sábado 12 de septiembre de 2026 · 9:00 - 11:00 a.m.

UNAB · Salón L51 · Bucaramanga

Organizan: BucaraTec · AWS User Group Bucaramanga · UNAB TEC

PASO 1

Spec inicial en el chat de Kiro

Se redacta el prompt de spec directamente en el chat de Kiro, pidiendo explícitamente que solo genere los requisitos (**requirements.md**) y que no escriba código todavía. El prompt define las secciones mínimas de la página (hero, agenda, ponentes, footer), detalla los ocho bloques horarios del evento y pide que cada requisito quede con un criterio de aceptación claro y verificable, dejando la aprobación del usuario como condición antes de pasar al diseño técnico.

PROMPT

Spec:

Antes de programar, quiero que generes solo los REQUISITOS (requirements.md) de una landing page para el evento "Kiro en modo constructor: specs, agentes y una web real", que se realiza el 12 de septiembre de 2026, de 9:00 a 11:00 a.m., en la UNAB, salón L51.

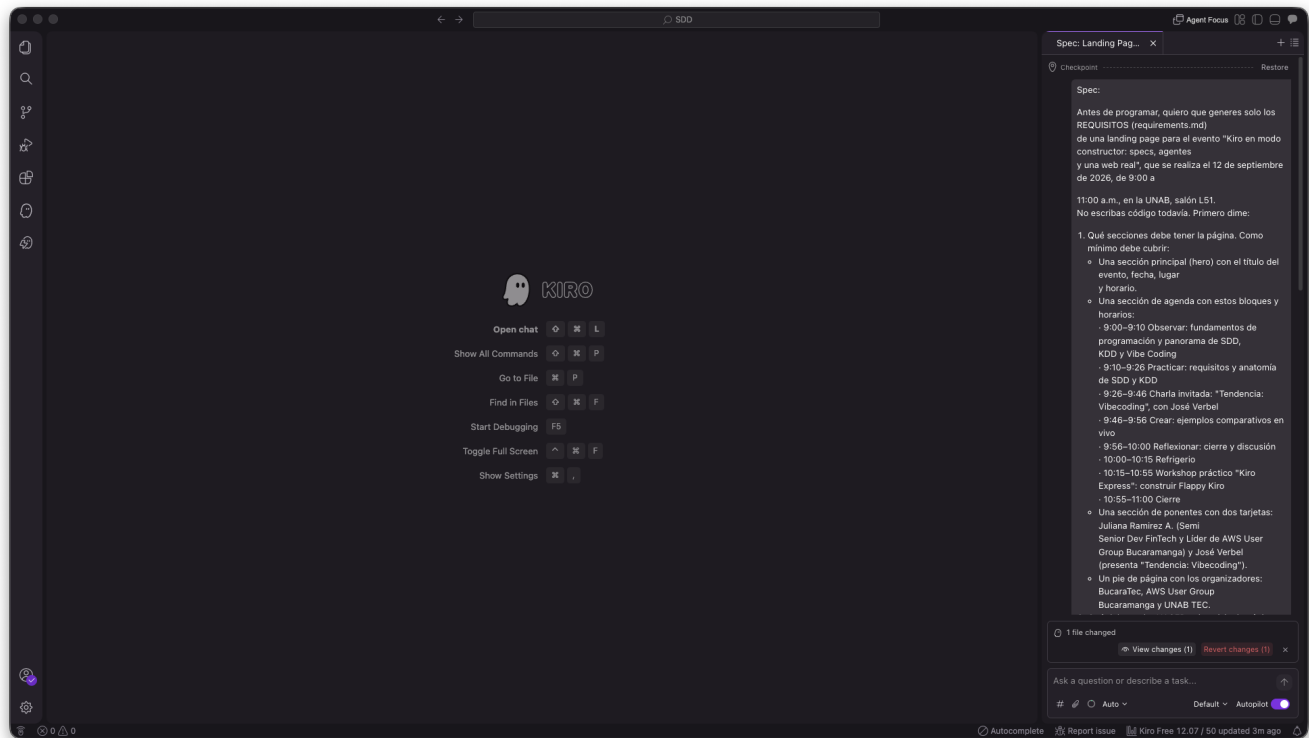
No escribas código todavía. Primero dime:

1. Qué secciones debe tener la página. Como mínimo debe cubrir:

- Una sección principal (hero) con el título del evento, fecha, lugar y horario.
- Una sección de agenda con estos bloques y horarios:
 - 9:00-9:10 Observar: fundamentos de programación y panorama de SDD, KDD y Vibe Coding
 - 9:10-9:26 Practicar: requisitos y anatomía de SDD y KDD
 - 9:26-9:46 Charla invitada: "Tendencia: Vibecoding", con José Verbel
 - 9:46-9:56 Crear: ejemplos comparativos en vivo
 - 9:56-10:00 Reflexionar: cierre y discusión
 - 10:00-10:15 Refrigerio
 - 10:15-10:55 Workshop práctico "Kiro Express": construir Flappy Kiro
 - 10:55-11:00 Cierre
- Una sección de ponentes con dos tarjetas: Juliana Ramirez A. (Semi Senior Dev FinTech y Líder de AWS User Group Bucaramanga) y José Verbel (presenta "Tendencia: Vibecoding").
- Un pie de página con los organizadores: BucaraTec, AWS User Group Bucaramanga y UNAB TEC.

2. Qué debe poder HACER quien visite la página (requisitos funcionales), por ejemplo: ver la agenda completa, identificar quién habla en cada bloque, y que la página se vea bien tanto en computador como en celular (requisito no funcional de responsividad).

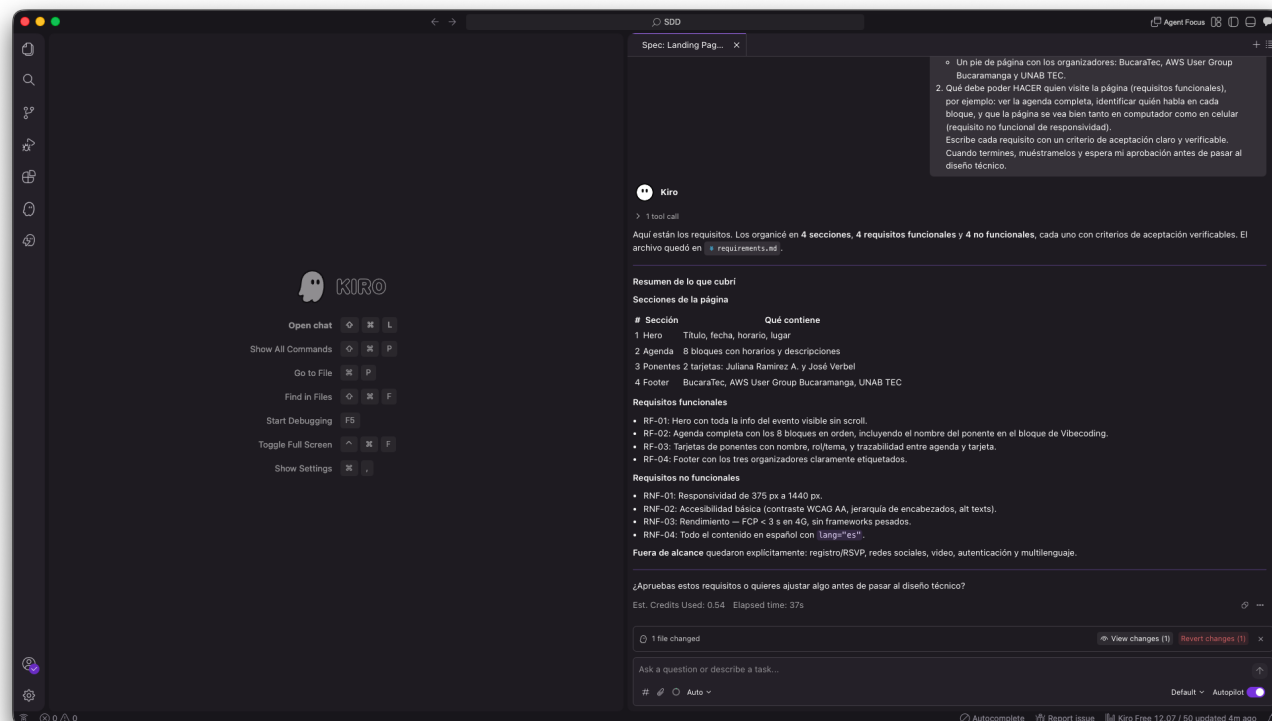
Escribe cada requisito con un criterio de aceptación claro y verificable. Cuando termines, muéstramelos y espera mi aprobación antes de pasar al diseño técnico.



PASO 2

Secciones, requisitos funcionales y no funcionales

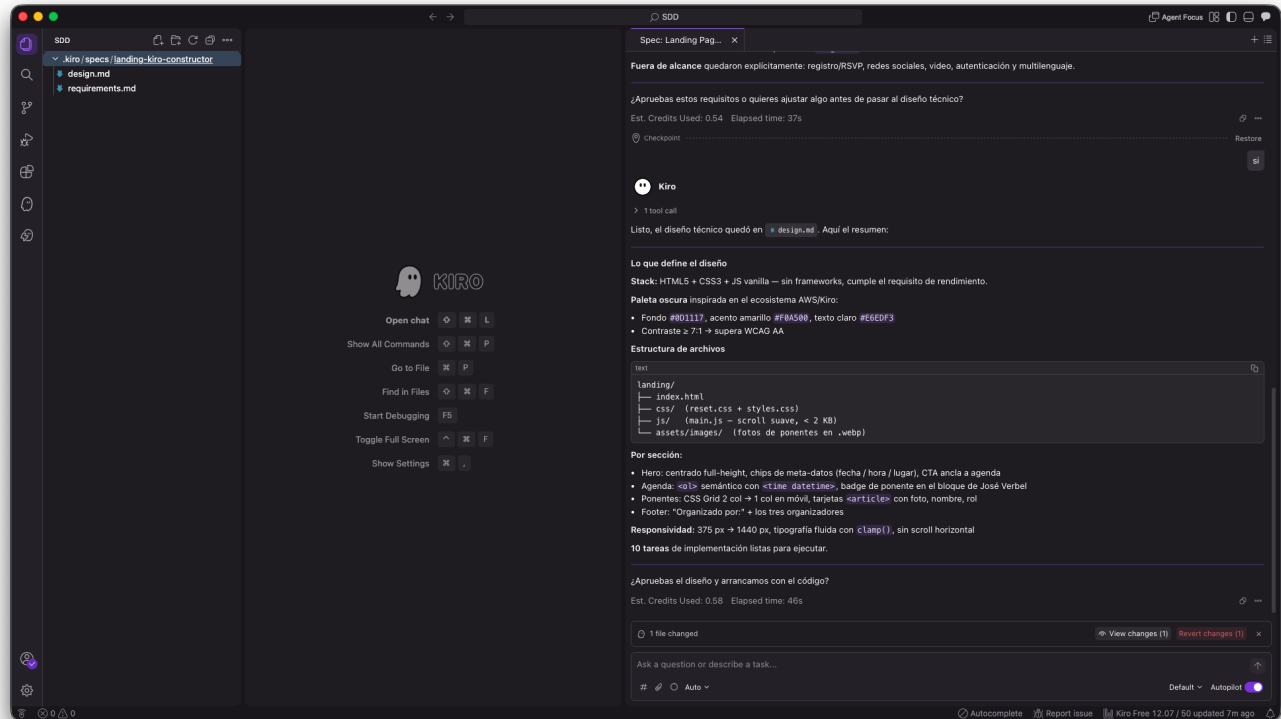
Kiro responde organizando la propuesta en 4 secciones (hero, agenda, ponentes, footer), 4 requisitos funcionales (RF-01 a RF-04) y 4 no funcionales (RNF-01 a RNF-04), cada uno con su criterio de aceptación verificable. También deja explícito qué queda fuera de alcance (registro/RSVP, redes sociales, video, autenticación y multilenguaje) y cierra pidiendo aprobación antes de continuar.



PASO 3

Archivos generados por Kiro

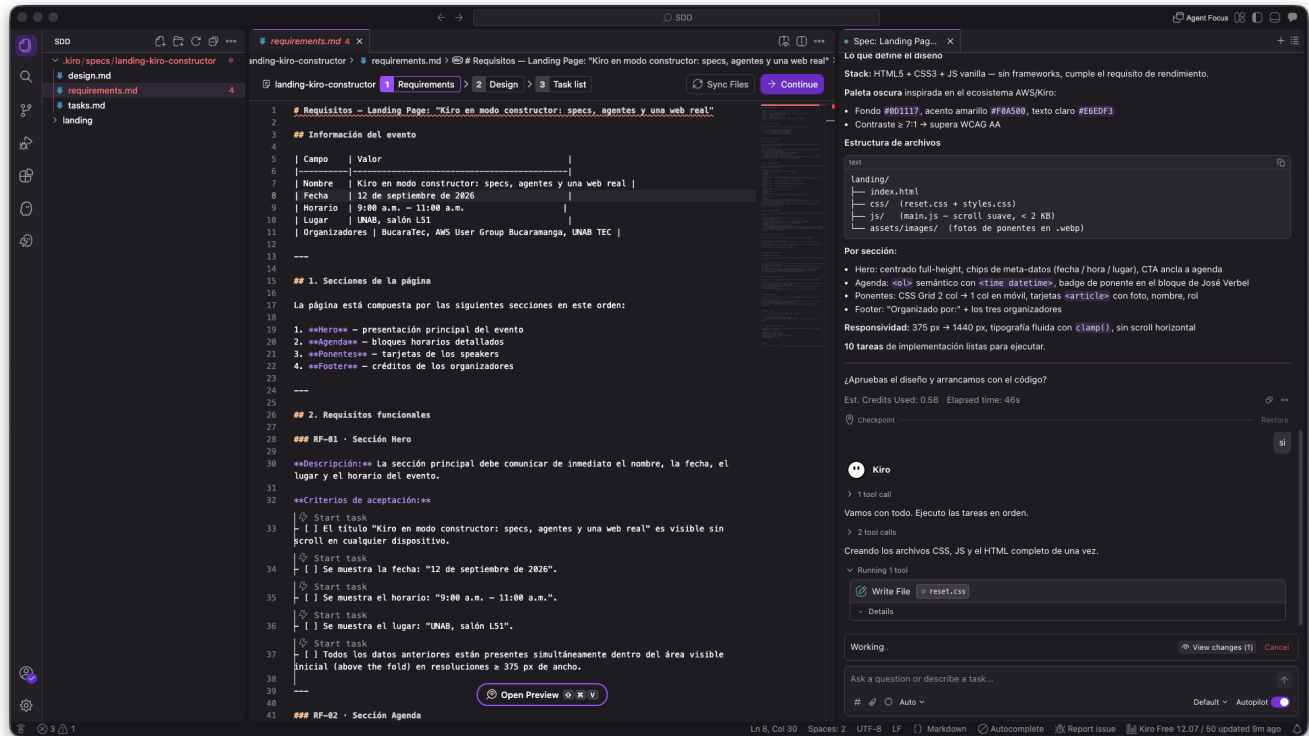
Al aprobar la propuesta, Kiro va indicando los siguientes pasos y, a medida que avanza, crea nuevos archivos dentro de `.kiro/specs/landing-kiro-constructor/`, visibles en el panel izquierdo. Primero aparece `requirements.md` y luego `design.md`, con el diseño técnico: stack (HTML5 + CSS3 + JS vanilla), paleta oscura inspirada en AWS/Kiro, estructura de carpetas y desglose por sección, además de las 10 tareas de implementación listas para ejecutar.



PASO 4

Visualización de requirements.md

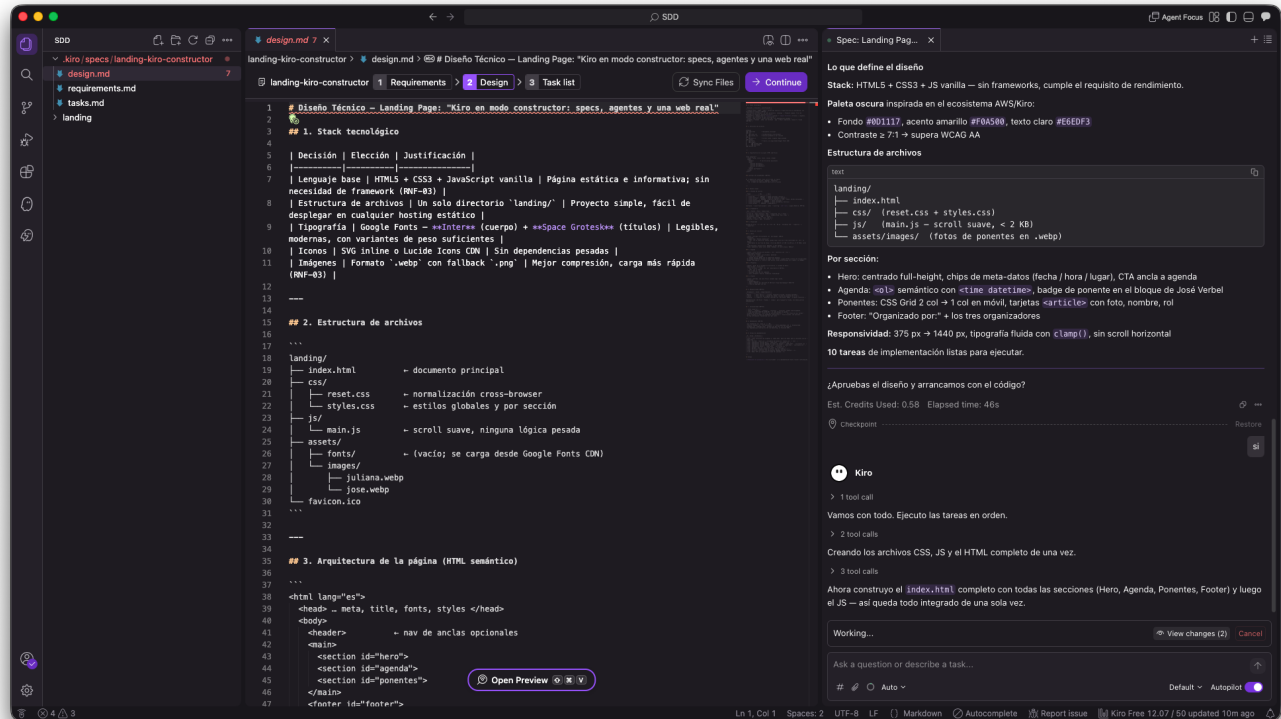
Se visualizan todos los requisitos que han sido predefinidos según la propuesta de Kiro: la tabla con los datos del evento, el orden de las 4 secciones y cada requisito funcional (RF-01 en adelante) con su descripción y sus criterios de aceptación redactados como checklist verificable. Cada criterio aparece además vinculado a su propia tarea ejecutable (“Start task”), mostrando la trazabilidad completa entre requisito, tarea y código.



PASO 5

Visualización de design.md

El archivo de diseño técnico pasa de “qué construir” a “cómo construirlo”: cada decisión (stack vanilla, un solo directorio, tipografías Google Fonts, imágenes en .webp) queda justificada y ligada de vuelta a un requisito no funcional. Luego detalla la estructura de archivos con comentarios línea por línea y arranca el esqueleto HTML semántico con las secciones hero, agenda, ponentes y footer, listo para que Kiro empiece a escribir el código final.

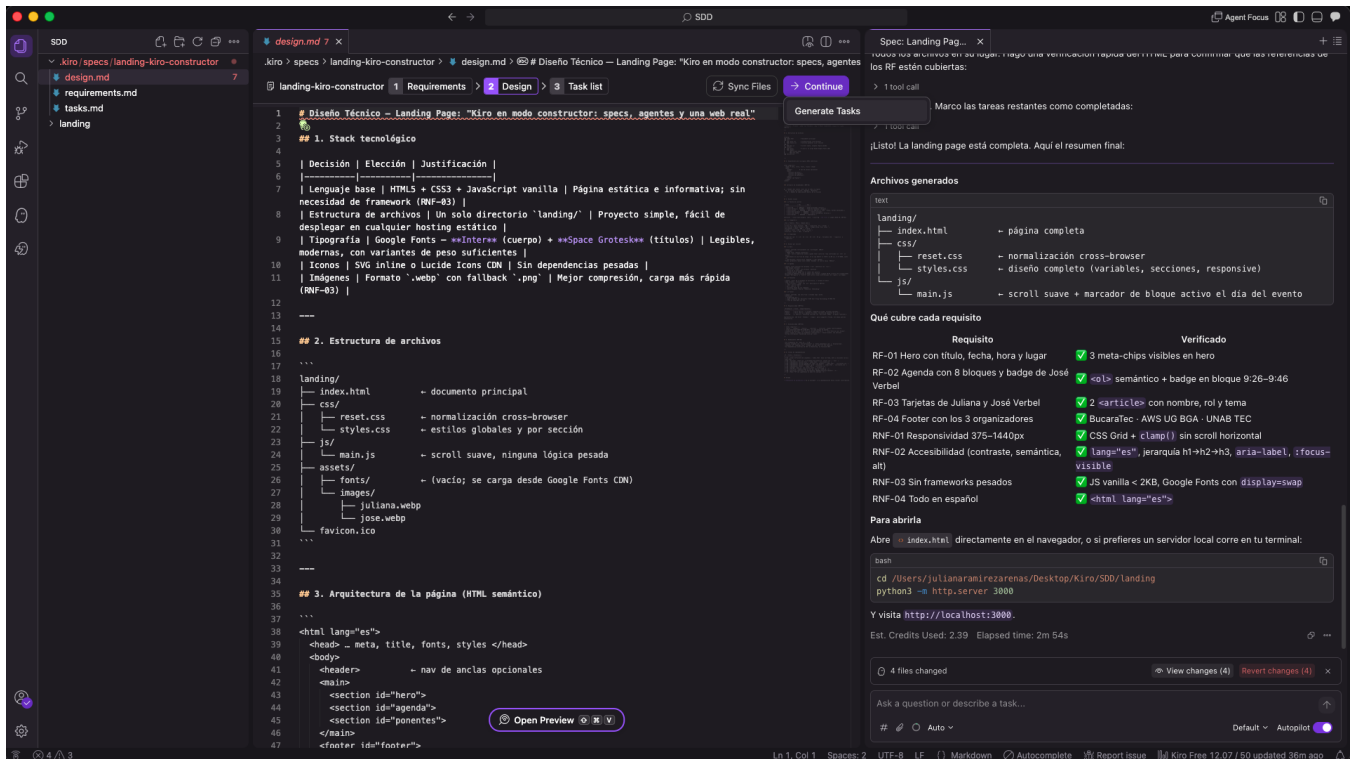


PASO 6

Generación y ejecución del plan de tareas

Con el diseño técnico aprobado, la barra superior del spec (**Requirements** → **Design** → **Task list**) permite avanzar al último eslabón: al hacer clic en **Continue**, Kiro ofrece la opción **Generate Tasks** para desglosar el diseño en un plan de tareas ejecutable (**tasks.md**). Una vez generadas y ejecutadas, Kiro reporta el resultado final: los archivos completos (index.html, css/reset.css, css/styles.css, js/main.js) y una tabla que verifica cada requisito funcional y no funcional contra lo implementado —por ejemplo, RF-02 con el **** semántico y el badge en el bloque de José Verbel, o RNF-03 con JS vanilla menor a 2KB—, cerrando el ciclo completo de trazabilidad de SDD: requisito → diseño → tarea → código verificado.

Antes de continuar hay que esperar a que Kiro termine de ejecutar el plan de tareas: mientras trabaja, el panel inferior del chat siempre muestra la palabra “Working...”, y solo cuando esa palabra desaparece se puede asumir que el proceso finalizó.

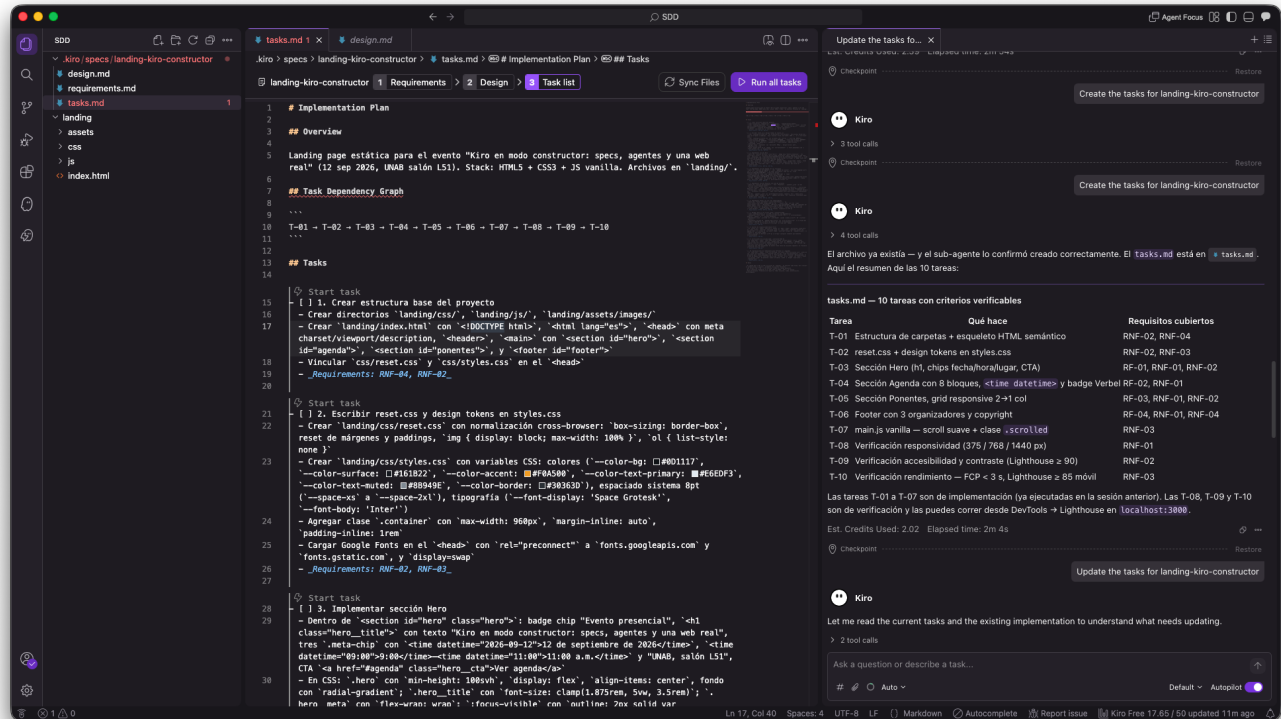


PASO 7

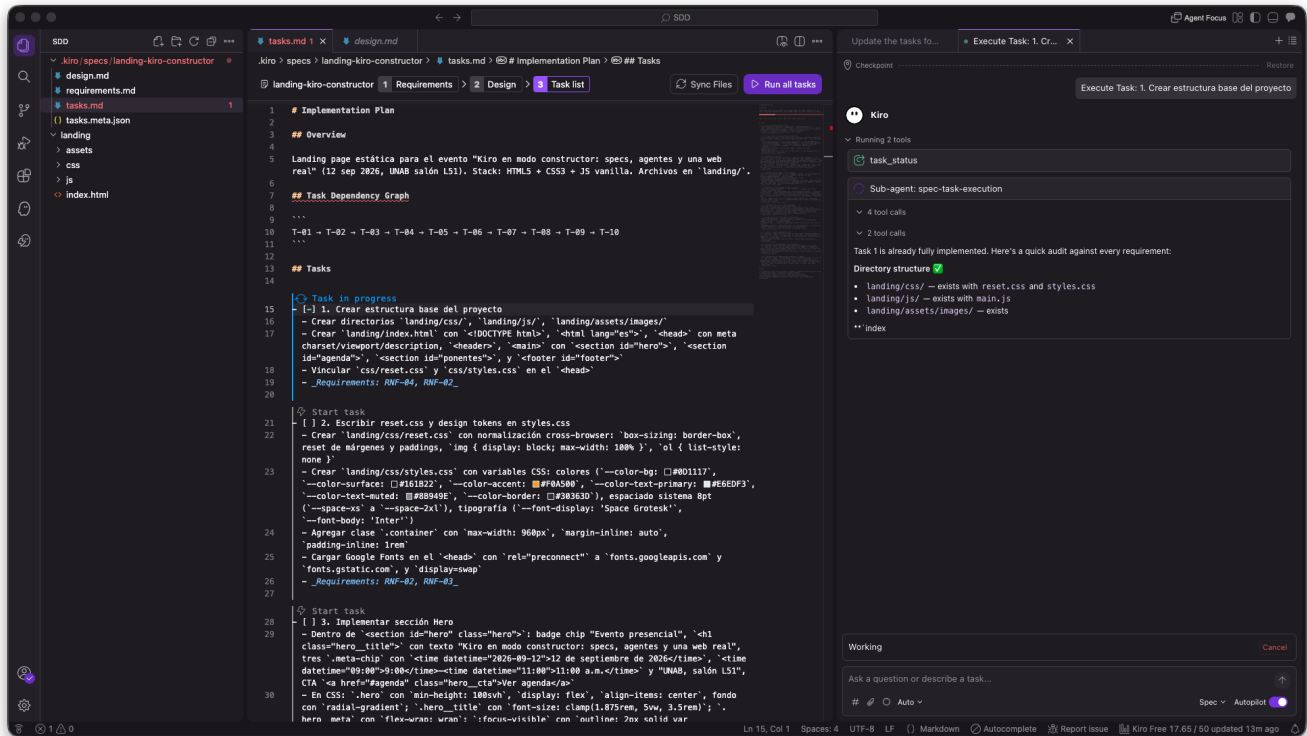
Visualización de tasks.md y ejecución controlada

El archivo **tasks.md** desglosa el diseño en un plan de implementación con un grafo de dependencias (T-01 → T-02 → ... → T-10) y 10 tareas con criterios verificables, cada una vinculada a los requisitos que cubre —por ejemplo, T-03 (sección Hero) cubre RF-01, y T-08 (verificación de responsividad) cubre RNF-01—. En la parte superior del panel está el botón **Run all tasks**, que lanza el plan completo de una sola vez; pero cada tarea tiene además su propio botón **Start task**, que permite ejecutarla de forma individual.

Se aconseja ejecutar las tareas una a una con Start task en lugar de usar Run all tasks: esto ayuda a evitar un mayor consumo de tokens de Kiro y también reduce el tiempo de ejecución de cada paso.



Detalle: cada tarea incluye su propio botón Start task.



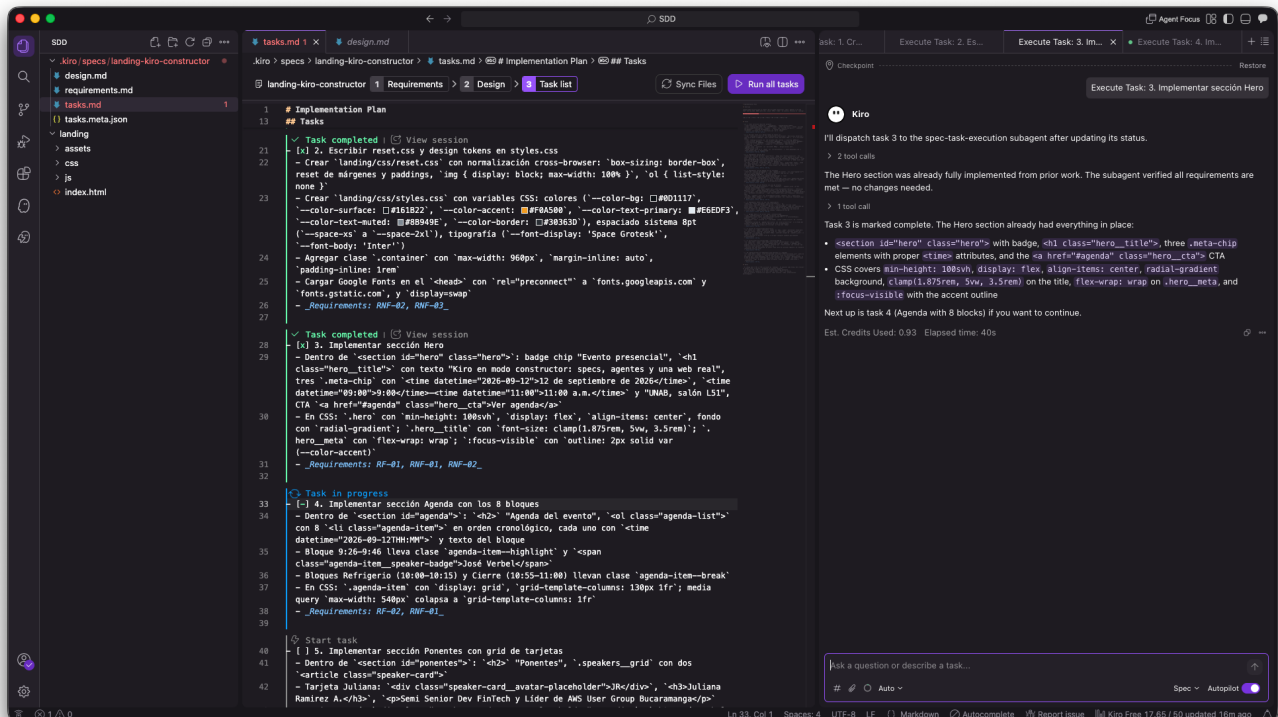
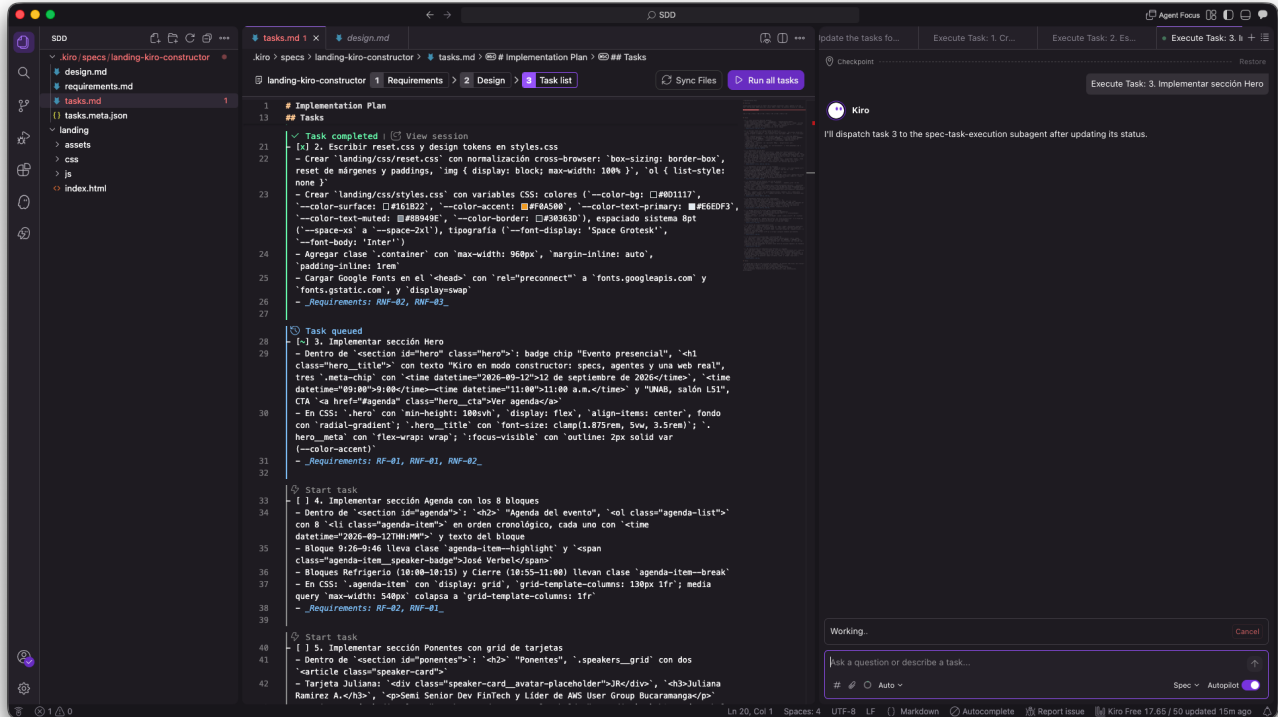
El botón **Run all tasks** queda arriba, junto al selector de fase; aquí la tarea 1 se ejecuta de forma individual y el panel inferior muestra **“Working”** mientras el sub-agente trabaja.

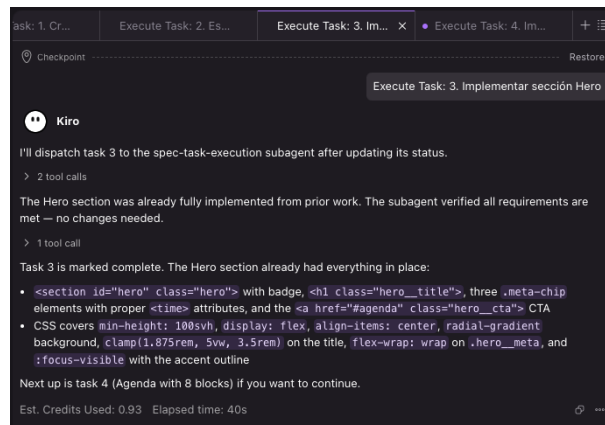
PASO 8

Cada tarea corre en su propia sesión de chat

En **tasks.md**, cada tarea va cambiando de estado a medida que avanza: primero aparece como **Task queued** ([~]), luego **Task in progress** ([-]) y finalmente **Task completed** ([x]). Cada una de estas ejecuciones abre su propia pestaña de chat en el panel derecho del aplicativo —**Execute Task: 1, Execute Task: 2, Execute Task: 3...**—, y esa sesión nueva tiene contexto únicamente de esa tarea puntual, no del resto del historial. Por ejemplo, al ejecutar la tarea 3 (sección Hero), Kiro revisa solo esa sección, confirma que ya cumplía los requisitos y cierra sugiriendo continuar con la tarea 4.

Cada escenario ya desarrollado también admite correcciones si hacen falta, pero deben pedirse dentro de esa misma sesión —por ejemplo, en la pestaña **Execute Task: 3** si el ajuste es sobre la sección Hero—, ya que, como se mencionó, solo esa tarea conserva el contexto de lo que se ha ejecutado.





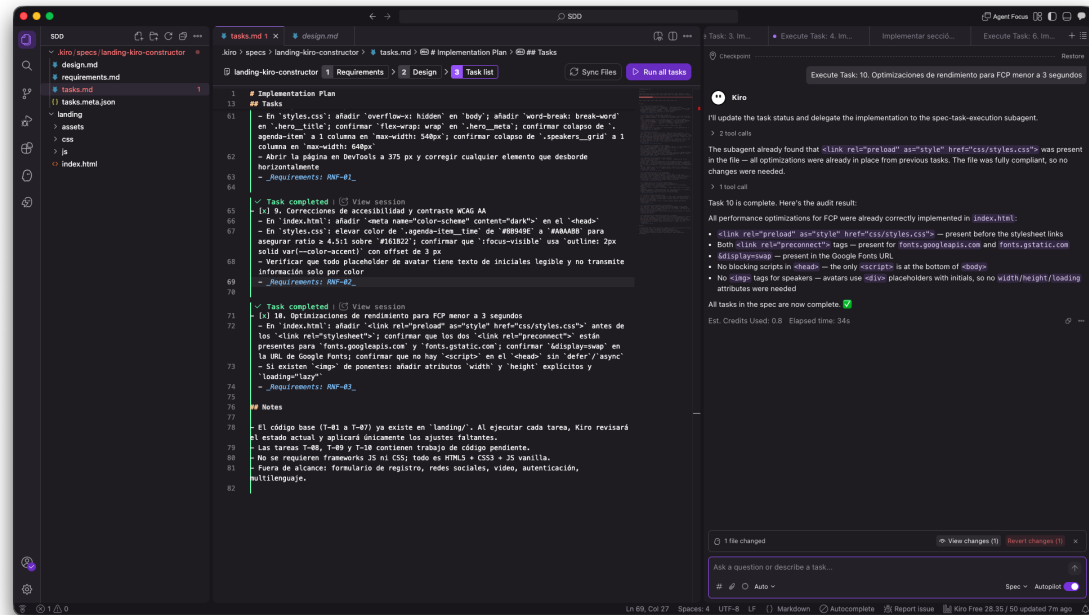
Detalle: la sesión “Execute Task: 3” entrega su propio resumen, aislado de las demás tareas.

PASO 9

Todas las tareas ejecutadas: cierre del plan

Las últimas tareas también quedan marcadas **Task completed** con su propia auditoría: la 9 revisa accesibilidad y contraste WCAG AA (color-scheme, contraste $\geq 4.5:1$, focus-visible), y la 10 revisa el rendimiento para FCP menor a 3 segundos (preload del CSS, preconnect a las fuentes, display=swap, sin scripts bloqueantes). Al final de **tasks.md**, la sección **Notes** deja constancia de que el código base (T-01 a T-07) ya existía y que solo T-08, T-09 y T-10 tenían trabajo pendiente, además de confirmar que no se requieren frameworks y qué quedó fuera de alcance. Con la tarea 10 verificada, Kiro cierra el plan completo: “All tasks in the spec are now complete.”

En el panel izquierdo del explorador ya se identifican todos los elementos de la aplicación creados: la carpeta landing/ con sus subcarpetas assets/, css/ y js/, además del archivo index.html —la estructura completa que design.md había planeado desde el principio.



PASO 10

Resultado final: la landing page renderizada

Al abrir `index.html` en el navegador se confirma el resultado: el hero con el badge “Evento presencial”, título y subtítulo, los chips de fecha, horario y lugar, y el botón “Ver agenda”, todo bajo la paleta oscura con acento naranja definida en `design.md`. Esta versión podría mejorarse agregando nuevos requisitos; sin embargo, lo que se puede notar es que se trata de una versión controlada: se ejecutó exactamente según lo dispuesto en los RF y RNF escritos desde el principio, cerrando de extremo a extremo el ciclo de SDD.

